SW407 · SISTEMAS OPERATIVOS · SEMANA 3

Procesos e Hilos

La cocina del sistema operativo: cómo se organiza el trabajo dentro de una computadora

Escuela de Ingeniería de Software

Facultad de Ingeniería Industrial y de Sistemas — UNI

Docente: Mg. Ing. José Carlos, García La Riva · 2026-II

Logros de la sesión

1. **Explicar el concepto de proceso: su estado, contexto y ciclo de vida**

   Nuevo, listo, en ejecución, bloqueado y terminado.

2. **Diferenciar la creación y terminación de procesos mediante fork() y exec()**

   Comprendiendo qué ocurre internamente en cada llamada.

3. **Distinguir un hilo (thread) de un proceso**

   Identificando qué comparten y qué mantienen propio.

4. **Comparar los modelos de hilos a nivel de usuario y a nivel de kernel**

   Y su impacto en la ejecución paralela.

EL ESCENARIO

Bienvenidos a la cocina del sistema operativo

Imagina un restaurante muy concurrido. Cada vez que llega un pedido, la cocina le asigna todo lo necesario para prepararlo: una tabla de trabajo, ingredientes, utensilios y un cocinero a cargo. Ese pedido completo —con todo lo que necesita para existir— es exactamente lo que el sistema operativo llama un PROCESO.

Pero dentro de ese mismo pedido, el cocinero principal puede pedir ayuda: un segundo cocinero corta los vegetales mientras un tercero prepara la salsa, todos usando la misma tabla, el mismo fuego y los mismos ingredientes de ese pedido. Esos cocineros que colaboran dentro de un mismo pedido son los HILOS (threads) de nuestro sistema.

Hoy vamos a entender cómo nace un pedido, por qué etapas pasa hasta llegar a la mesa, y qué diferencia a un cocinero que trabaja solo de un equipo que colabora en la misma orden.

¿Qué es un proceso?

Un proceso es un programa en ejecución: no basta con tener la receta escrita, hace falta que alguien la esté cocinando en este momento.

1

Programa vs. proceso

El programa es la receta escrita en un libro de cocina (el código en disco); el proceso es esa receta siendo preparada ahora mismo, con ingredientes reales, un cocinero asignado y un punto exacto de avance.

2

Espacio de memoria propio

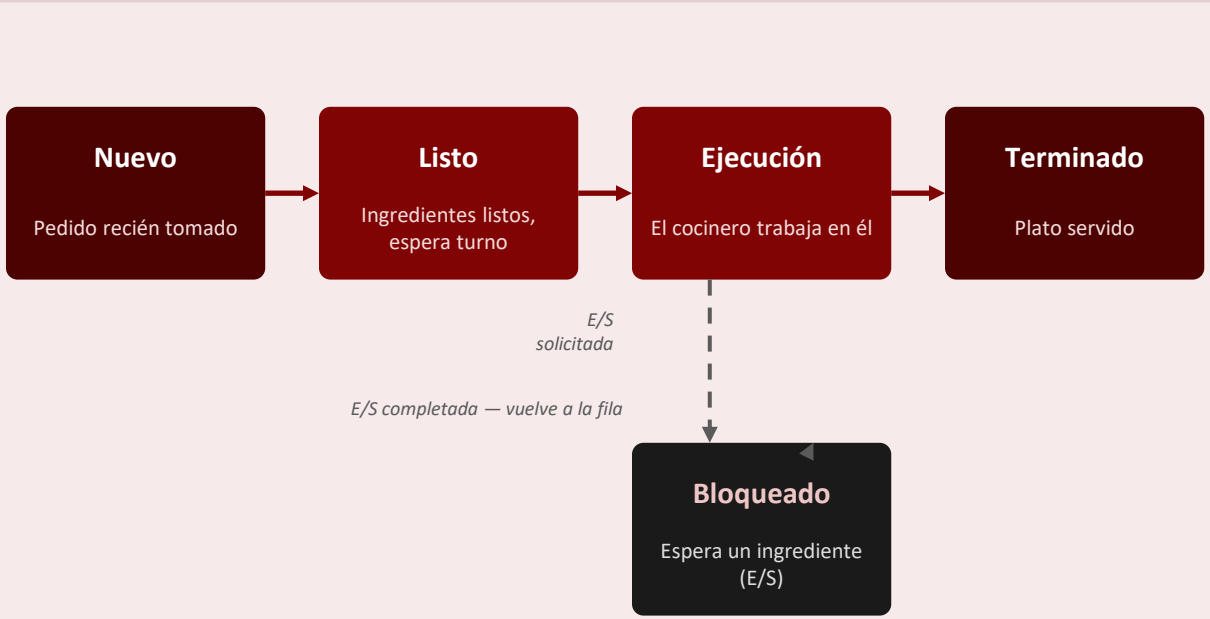
Cada proceso tiene su propia "cocina": su espacio de memoria aislado (código, datos, pila y montículo), que otros procesos no pueden ver ni modificar directamente.

3

Recursos asignados

CPU, archivos abiertos, conexiones de red y otros recursos que el sistema operativo reserva para ese proceso mientras existe.

Los cinco estados de un proceso



- **Nuevo → Listo**

  El proceso se crea y espera a que el planificador le asigne un turno de CPU.

- **Listo → Ejecución → Terminado**

  Recibe la CPU, ejecuta sus instrucciones y finaliza al completar su tarea.

- **El ciclo Bloqueado**

  Si necesita esperar un recurso (E/S), cede la CPU y regresa a "Listo" solo cuando esa espera termina.

El contexto del proceso: la comanda de cocina

El sistema operativo guarda toda la información de cada proceso en una estructura llamada PCB (Process Control Block) — como la comanda que acompaña a cada pedido en la cocina.

ID — **Identificador (PID)**
El número de orden que distingue a este pedido de todos los demás.

Reg — **Registros de CPU**
Las medidas y cantidades que el cocinero tenía en mente justo antes de una interrupción.

St — **Estado actual**
Si está nuevo, listo, en ejecución, bloqueado o terminado.

Mem — **Punteros de memoria**
La ubicación de la tabla de trabajo y los ingredientes asignados a este pedido.

PC — **Contador de programa**
La línea exacta de la receta en la que se quedó el cocinero.

E/S — **Archivos y dispositivos abiertos**
Qué proveedores o almacenes tiene reservados este pedido en este momento.

Creando y reemplazando procesos: fork() y exec()

En sistemas tipo UNIX/Linux, un proceso no aparece de la nada: siempre lo crea otro proceso ya existente, usando estas dos llamadas al sistema.

1

fork() — clonar al cocinero

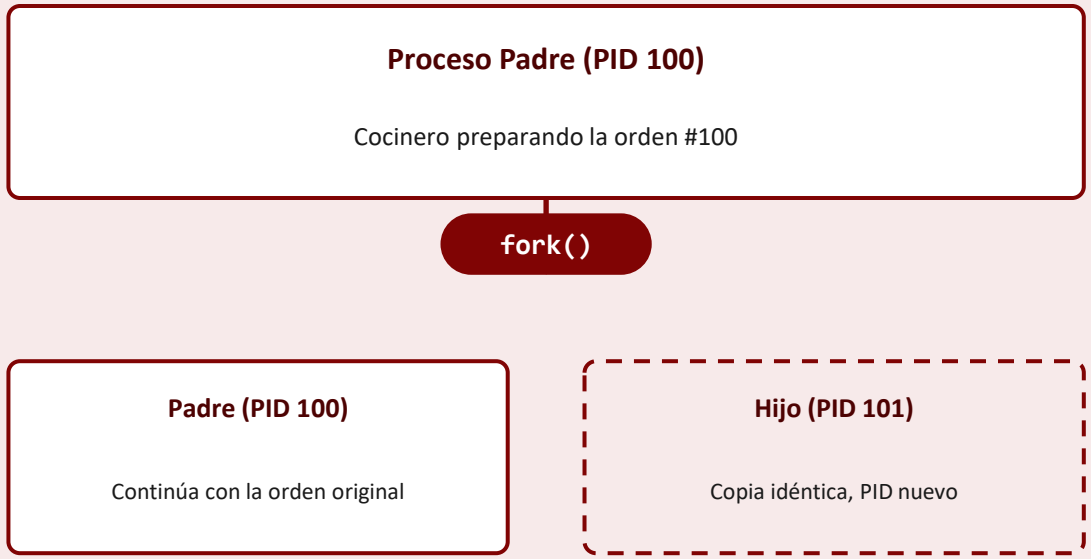
El proceso actual (padre) crea una copia casi idéntica de sí mismo (hijo): mismo código, mismas variables, mismo punto de la receta. A partir de ahí, ambos continúan de forma independiente.

2

exec() — cambiar de receta

El proceso hijo descarta por completo su receta actual y carga un programa totalmente distinto en su lugar, conservando el mismo PID pero con código e instrucciones nuevas.

fork(): un cocinero se duplica



- **Qué se copia**

  El código, las variables, los archivos abiertos y el punto exacto de ejecución del proceso padre.

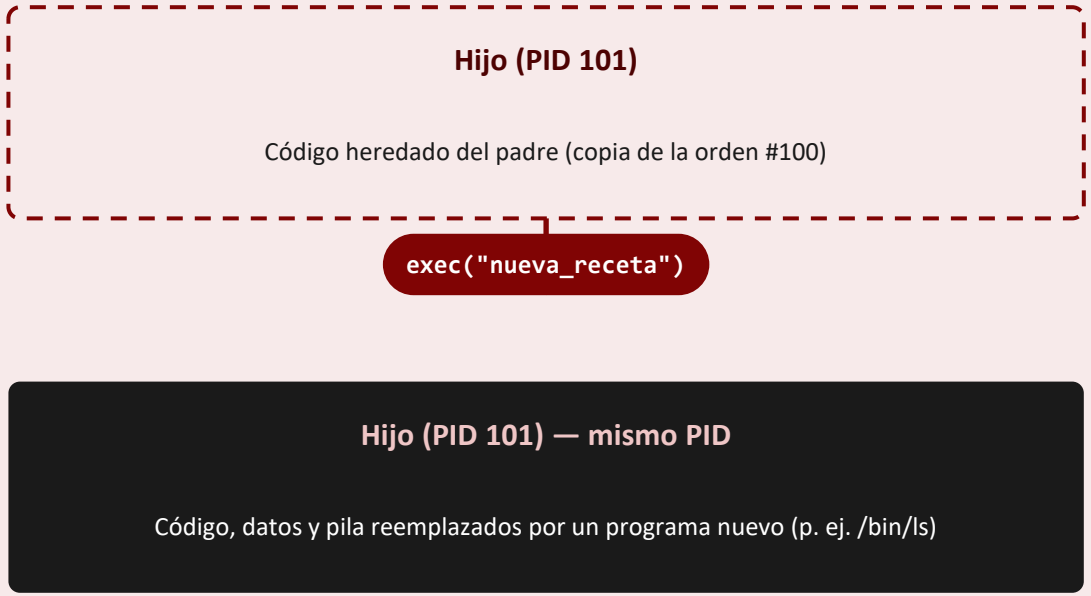
- **Qué cambia**

  El hijo recibe un PID (identificador) propio y distinto al de su padre.

- **Valor de retorno**

  fork() devuelve 0 al hijo y el PID del hijo al padre — así cada uno sabe "quién es" después de la llamada.

exec(): el ayudante cambia de receta



- **Lo que NO cambia**

  El PID del proceso se mantiene igual: sigue siendo el mismo "puesto de trabajo" en la cocina.

- **Lo que sí cambia**

  Todo el contenido: el código en ejecución, las variables y la pila pasan a ser los del nuevo programa.

- **Patrón habitual**

  fork() + exec() juntos son la forma clásica en que un shell (como bash) ejecuta cada comando que escribes.

De un cocinero a un equipo de cocineros

Volvamos a nuestra orden #100. El cocinero principal se da cuenta de que puede avanzar más rápido si no hace todo solo: le pide a un segundo cocinero que corte los vegetales y a un tercero que prepare la salsa, mientras él se encarga de la proteína principal.

Los tres trabajan al mismo tiempo, sobre la misma tabla, con los mismos ingredientes y el mismo fuego de esa orden — no son pedidos distintos, son colaboradores dentro del mismo pedido.

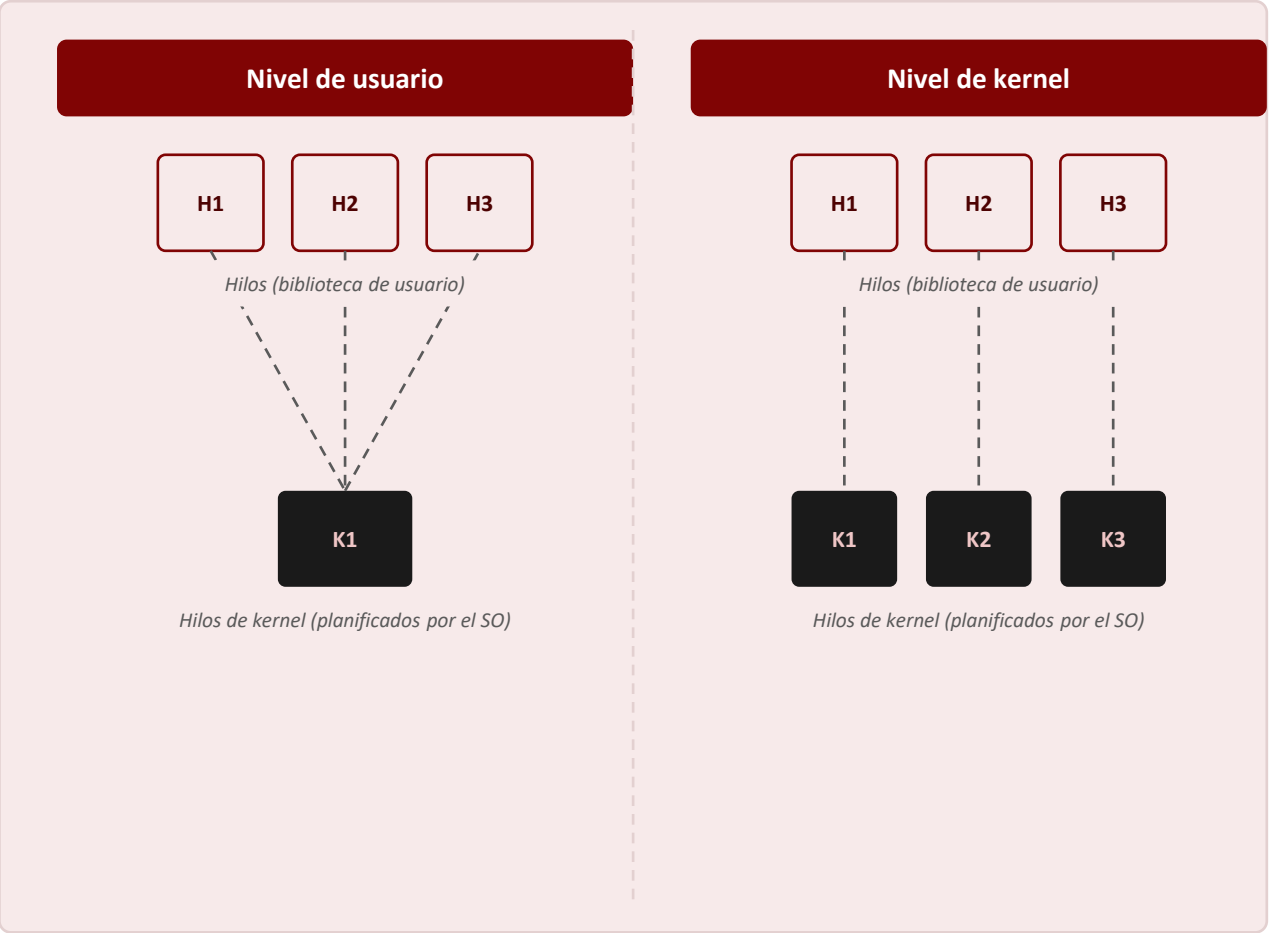
Eso es exactamente un HILO (thread): la unidad más pequeña de ejecución dentro de un proceso, que comparte su memoria y sus recursos con los demás hilos de ese mismo proceso, pero mantiene su propio punto de avance dentro de la receta.

COMPARACIÓN

Procesos vs. hilos

Criterio	Proceso	Hilo (thread)
Analogía de cocina	Un pedido completo, con su propia tabla e ingredientes	Un cocinero colaborando dentro de un mismo pedido
Espacio de memoria	Propio y aislado del de otros procesos	Compartido con los demás hilos del mismo proceso
Costo de creación	Alto: se duplica todo el contexto (fork)	Bajo: reutiliza el contexto del proceso existente
Comunicación	Requiere mecanismos de IPC (pipes, sockets, etc.)	Directa, a través de la memoria compartida
Efecto de un fallo	Un proceso caído no afecta a otros procesos	Un hilo con error puede afectar a todo el proceso

Hilos de usuario vs. hilos de kernel



- **Usuario (muchos a uno)**

  Un cocinero organiza mentalmente varias sub-tareas sin avisar al gerente; si una se bloquea, todas esperan.

- **Kernel (uno a uno)**

  El gerente (SO) conoce y programa a cada cocinero por separado: pueden trabajar en paralelo real en distintas hornillas (núcleos).

- **Costo vs. paralelismo real**

  Los hilos de usuario son más ligeros de crear; los de kernel aprovechan mejor varios núcleos de CPU.

Simulando cocineros con threading en Python

python

```python
import threading

def cocinar(tarea):
    print(f"Preparando: {tarea}")

tareas = ["cortar", "freír", "salsa"]
hilos = []
for t in tareas:
    h = threading.Thread(target=cocinar, args=(t,))
    hilos.append(h)
    h.start()          # cada hilo empieza a trabajar

for h in hilos:
    h.join()           # esperar a que todos terminen

print("Orden #100 lista para servir")
```

Cada objeto Thread representa a un cocinero; start() lo pone a trabajar y join() espera a que termine antes de servir el plato.

- **threading.Thread**

  Crea un nuevo hilo que ejecutará la función indicada de forma concurrente con los demás.

- **start() vs. join()**

  start() inicia el trabajo del hilo; join() bloquea al hilo principal hasta que ese hilo termine.

- **Memoria compartida**

  Todos los hilos ven las mismas variables globales — por eso conviene tener cuidado con el acceso simultáneo (lo veremos en Sincronización, semana 5).

APLICACIÓN

¿Dónde vemos procesos e hilos todos los días?

Navegadores web

Cada pestaña suele ser un proceso independiente (si una falla, no se cae todo el navegador); dentro de cada pestaña, varios hilos manejan red, renderizado e interacción.

Servidores web

Atienden muchas solicitudes de clientes al mismo tiempo usando procesos o hilos independientes por conexión.

Videojuegos

Un proceso principal con hilos dedicados a física, gráficos, audio e inteligencia artificial, todos avanzando en paralelo.

Editores de texto e IDEs

Un hilo mantiene la interfaz respondiendo mientras otro guarda el archivo o revisa la ortografía en segundo plano.

TALLER EN CLASE

Manos a la obra: procesos e hilos

Trabaja en pareja. Documenta cada resultado con una captura de pantalla para la entrega.

01 En C, usa fork() para crear un proceso hijo. Imprime un mensaje distinto en el padre y en el hijo, incluyendo su PID.

02 Modifica el ejercicio anterior para que el proceso hijo use exec() y ejecute el comando ls -l en lugar de continuar con su propio código.

03 En Python, crea 3 hilos con threading que simulen a tres cocineros preparando tareas distintas de una misma orden.

04 Documenta el PID del proceso padre y del hijo, y el orden en que terminan los hilos — ¿siempre termina primero el mismo?

CIERRE

Conclusiones

1. Un proceso es una receta siendo cocinada en este momento: tiene memoria propia, recursos asignados y pasa por estados definidos (nuevo, listo, ejecución, bloqueado, terminado).

2. fork() clona un proceso existente; exec() reemplaza el código de un proceso por uno nuevo — juntos son la base de cómo un shell ejecuta comandos.

3. Un hilo es la unidad de ejecución más pequeña dentro de un proceso: comparte memoria con sus hermanos, lo que lo hace más liviano pero también más sensible a errores compartidos.

4. Elegir entre hilos de usuario y de kernel es un balance entre ligereza de creación y aprovechamiento real de varios núcleos de CPU.

Referencias bibliográficas

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley — Capítulos 3 (Processes) y 4 (Threads).

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.

Python Software Foundation (2026). threading — Thread-based parallelism. docs.python.org/3/library/threading.html

The Linux man-pages project (2026). fork(2) y execve(2) — manual pages. man7.org

Universidad Nacional de Ingeniería (2026). Sílabo SW407 — Sistemas Operativos, Escuela de Ingeniería de Software.